

1.SGLang 的 RadixAttention / RadixCache 解决什么问题？

RadixCache 解决多轮请求中公共前缀的重复计算问题

(多轮对话中，许多问题的前缀一致 重复计算浪费算力)

而 RadixCache 利用基数树组织 KV Cache

将无分支的链压缩成一个节点 当有新的请求

从根节点遍历树 找到最长匹配前缀 命中部分复用算好的 KV Cache

只需计算新的部分 新部分作为分支成为树的一部分(动态树)

长期不用的冷前缀 被回收 给新的词缀留位置

2.SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

Page-sized KV cache:prefill 计算完成后 将 KV 按页存储

逻辑上连续 物理上散落 更好利用空间

Prefix reuse: 新的请求 遍历基数树是否命中前缀

命中则跳过 复用计算好的 KV 计算新增部分

未命中走一遍 prefill 结果长到树上

3.vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

PagedAttention 将 KV Cache 切割成固定的页

并将连续的逻辑空间映射到离散的物理空间

按需分配 提高显存的利用率

联系：SGLang 的 page-sized KV Cache 思路和 PagedAttention 一致

差异：vLLM 保证存的下 SGLang 保证算的省

SGLang 比 vLLM 加重重复前缀的复用能力

